S.No: 1	Exp. Name: ***Print Hello World***	**Date: 2026-02-05**

Aim:

Ravi has joined his first programming class. Before learning advanced concepts, his instructor asks every student to check whether their compiler and development environment are set up correctly. To do this, Ravi is instructed to write a simple C++ program that displays the message **Hello World** on the output screen.

Write a C++ program that prints **Hello World** on the output screen.

Source Code:

main.cpp

```
#include<iostream>
using namespace std;
int main()
{
        cout<<"Hello World";
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Hello World

ID: 2503051050907
2025-2029-2A13-3
Parul University

S.No: 2	Exp. Name: ***Prime Number Analyzer***	**Date: 2026-02-12**

Aim:

You are developing a **number-crunching application** for a data analysis system. One of the core features of this application is to **identify prime numbers** within a given range. The system administrator provides an **upper limit n**, and the application must list **all prime numbers between 1 and n**. However, to maintain performance constraints, if **no prime numbers** exist within the range, the system should notify the user accordingly.

Write a C++ program to generate and display **all prime numbers between 1 and n**, where the user provides n.

Input Format:

- A single integer n (n represents the upper limit).

Output Format:

- If there are **no prime numbers** between 1 and n, print: No primes.
- If n is greater than 100, print **Invalid**.
- Otherwise, print all prime numbers between 1 and n (inclusive) on the **same line**, separated by spaces.

Constraints:

- $1 \leq n \leq 100$

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

prime.cpp

```
#include <iostream>
using namespace std;

int main() {
        int n;
        cin>>n;
        if(n>100) {
                cout<<"Invalid";
                return 0;
        }
        int found=0;
        for(int i=2; i<=n;i++){
                int count=0;
                for(int j=2;j<i;j++) {
                        if(i%j==0){
                                count++;
                                break;
                        }
                }
        if(count==0){
                cout<<i<<" ";
                found = 1;
        }
        }
        if(found == 0){
                cout<<"No primes";
        }
return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
10
2 3 5 7

Test Case - 2

ID: 2503051050907
2025-2029-2A13-3
Parul University

User Output
1
No primes

Test Case - 3
User Output
101
Invalid

ID: 2503051050907
2025-2029-2A13-3
Parul University

S.No: 3	Exp. Name: ***Fibonacci Series***	**Date: 2026-02-19**

Aim:

Write a C++ program to generate and print the Fibonacci series up to a given number n. The Fibonacci series is a sequence of numbers where each number is the sum of the two preceding ones, typically starting with 0 and 1.

Input Format:

- The program should prompt the user to enter an integer n.

Output Format:

- Print the Fibonacci series up to n in a single line, separated by a space.

Def: The Fibonacci series is a sequence where:

- The first two numbers are **0 and 1.**
- Each subsequent number is the **sum of the previous two.**

Note: Refer to the visible test cases for better understanding.

Source Code:

Fibonacci.cpp

```
#include<iostream>
using namespace std;
int main()
{
        int n;
        cin>>n;
        int a=0;
        int b=1;
        int c=0;
        cout<<a;
        while(c<=n){
        c=a+b;
        cout<<" "<<b;
        a=b;
        b=c;
        }
        cout<<endl;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
10
0 1 1 2 3 5 8

Test Case - 2
User Output
20
0 1 1 2 3 5 8 13

ID: 2503051050907
2025-2029-2A13-3
Parul University

S.No: 4	Exp. Name: ***Factorial of a Number***	**Date: 2026-02-19**

Aim:

A mathematics application needs to compute the factorial of a given number. To demonstrate recursive problem-solving, the factorial calculation must be implemented using recursion. Write a C++ program to calculate the factorial of a given number using recursion.

Input Format:

- First line contains an integer n, where n is the number whose factorial is to be computed.

Output Format:

- Print the factorial of the number n.

Recursive Logic Explanation:

- If n is 0 or 1, return 1
- Otherwise, return n * factorial(n - 1)

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

fact.cpp

```
#include <iostream>
using namespace std;

// Function to calculate factorial using recursion
int factorial(int n) {
        if(n==0 || n==1) {
                return 1;
        }
        else {
                return n*factorial(n-1 );// Recursive case
        }

}
int main() {
    int num;

    // Input the number
    cin >> num;

    // Call the factorial function and display the result
    cout << factorial(num) << endl;

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
24

Test Case - 2
User Output
5
120

S.No: 5	Exp. Name: ***Maximum & Minimum Values***	**Date: 2026-02-26**

Aim:

You are developing a data-processing utility where the user dynamically provides a list of integers. The program must determine and display the **maximum** and **minimum** values from the entered data. Write a C++ program that dynamically allocates memory for an integer array based on user input, accepts array elements, and displays the **largest** and **smallest** values in the array.

Input Format:

- Enter the size of the array: An integer representing the size of the array (arraySize)
- Enter <arraySize> integers: The elements of the array.

Output Format:

- Maximum value: followed by the maximum element
- Minimum value: followed by the minimum element.

Constraints:

- $1 \leq \text{arraySize} \leq 10^5$
- Array elements are integers such that: $-10^9 \leq \text{array}[i] \leq 10^9$.

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

MinMax.cpp

```
#include<iostream>
using namespace std;
int main()
{
        int arraySize;
        cout<<"Enter the size of the array: ";
        cin>> arraySize;
        int array[arraySize];
        cout<<"Enter "<<arraySize<<" integers:";
        for(int i=0;i<arraySize;i++) {
                cin>>array[i];
        }
        int max=array[0];
        int min=array[0];

        for(int i=0; i<arraySize;i++) {

                if(array[i] > max) {
                        max=array[i];
                }

                if(array[i] < min){
                        min =array[i];
                }
        }
        cout<<"\nMaximum value: "<<max<<"\n";
                cout<<"Minimum value: "<<min<<"\n";

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the size of the array:
5
Enter 5 integers:
2 1 4 5 3
Maximum value: 5

ID: 2503051050907
2025-2029-2A13-3
Parul University

```
Minimum value: 1
```

Test Case - 2
User Output
`Enter the size of the array:`
6
`Enter 6 integers:`
-20 -50 100 200 0 40
`Maximum value: 200`
`Minimum value: -50`

S.No: 6	Exp. Name: ***Arithmetic Operations***	**Date: 2026-02-26**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

A basic calculator feature is required in a small billing application. The system must read two float values from the user and display their addition, subtraction, multiplication, division results.

Input Format:

- The user enters two integer values separated by a space.

```
Enter two values :
```

Output Format:

- The program should display the following results based on input:

```
Addition of two values = <value>
Subtraction of two values = <value>
Multiplication of two values = <value>
Division of two values = <value>
```

Source Code:

Operations.cpp

```
#include <iostream>
using namespace std;

int main() {
float a,b,sum,mult,sub,div;
        cout<<"Enter two values : ";
        cin>>a>>b;

        cout<<"Addition of two values = "<<a+b;
        cout<<"\nSubtraction of two values = "<<a-b;
        cout<<"\nMultiplication of two values = "<<a*b;
        cout<<"\nDivision of two values = "<<a/b<<"\n";

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter two values :
23.56 12.78
Addition of two values = 36.34
Subtraction of two values = 10.78
Multiplication of two values = 301.097
Division of two values = 1.84351

Test Case - 2
User Output
Enter two values :
12.5 2.0
Addition of two values = 14.5
Subtraction of two values = 10.5
Multiplication of two values = 25
Division of two values = 6.25

S.No: 7	Exp. Name: ***Sort Numbers in Ascending Order***	**Date: 2026-02-26**

Aim:

A data processing system receives a list of integers that must be arranged in ascending order before further analysis. As a developer, your task is to implement a program that sorts the given numbers from **smallest to largest**.

Input Format:

- The first line contains an integer **N**, representing the number of elements.
- The second line contains **N space-separated integers**.

Output Format:

- Print the **sorted list of numbers in ascending order**, separated by spaces.

Constraints:

- $1 \leq N \leq 10^5$
- $-10^9 \leq \text{elements} \leq 10^9$

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

sort.cpp

```
#include<iostream>
using namespace std;
int main()
{
        int N;
        cin>>N;
        int num[N];

        for(int i=0;i<N;i++){
                cin>>num[i];
        }

        for(int i = 0; i <N;i++){
                for(int j = 0;j<N-i-1;j++){
                        if(num[j] > num[j+1]){
                                int temp = num[j];
                                num[j] = num[j+1];
                                num[j+1] = temp;
                        }
                }
        }
        for(int i = 0;i<N;i++){
                cout<<num[i]<<" ";
        }
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
9 5 3 2 8
2 3 5 8 9

Test Case - 2
User Output

ID: 2503051050907
2025-2029-2A13-3
Parul University

3 6 5 1
1 3 5 6

S.No: 8	Exp. Name: ***Even or Odd Check***	**Date: 2026-02-26**

Aim:

A software utility needs to determine whether a given number entered by the user is even or odd. This helps in validating inputs for further numerical processing in the system. Write a C++ program that reads an integer and checks whether it is **even** or **Odd** using an if-else statement.

Input Format:

- An integer N

Output Format:

- Print "Even" if the number is even.
- Print "Odd" if the number is odd.

Constraints:

- $-10^9 \leq N \leq 10^9$.
- N is an integer input.

Source Code:

main3.cpp

```
#include<iostream>
using namespace std;
int main()
{
        int N;
        cin>>N;

if(N%2 == 0){
        cout<<"Even";
}
        else {
        cout<<"Odd";
                }

return 0;
}
```

Execution Results - All test cases have succeeded!

ID: 2503051050907
2025-2029-2A13-3
Parul University

Test Case - 1
User Output
4
Even

Test Case - 2
User Output
1
Odd

S.No: 9	Exp. Name: ***Program that illustrates the difference between call by value and call by reference***	**Date: 2026-02-27**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

Write a C++ program that demonstrates the concepts of call by value and call by reference using functions.

Your task is to write two functions:

- **swapByValue(int x, int y)**
- **swapByReference(int &x, int &y)**

The main function should do the following:

- Prompt the user to enter two integer values for **a** and **b**.
- Call a function to swap the values of **a** and **b** using call by value.
- Print the values of **a** and **b** after the first swap.
- Call another function to swap the values of **a** and **b** using call by reference (using references).
- Print the values of **a** and **b** after the second swap.

Input Format:

- The program prompts the user to enter two integer values separated by a space.

Output Format:

- In the first line, print the values of **a** and **b** after the first swap, separated by a space (using call by value)
- In the second line, print the values of **a** and **b** after the second swap (using call by reference)

Source Code:

swapNumbers.cpp

```
#include <iostream>
using namespace std;

// Function to swap values using call by value
void swapByValue(int x, int y) {

int temp=x;
x=y;
        y=10;


}
// Function to swap values using call by reference
void swapByReference(int &x, int &y) {
int temp=x;
        x=y;
        y=temp;



}
int main() {
    int a, b;
    cin >> a >> b;

    // Swap using call by value
    swapByValue(a, b);
    cout << a << " " << b << endl;

    // Swap using call by reference
    swapByReference(a, b);
    cout << a << " " << b << endl;

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

20 10
20 10
10 20

Test Case - 2
User Output
40 30
40 30
30 40

S.No: 10	Exp. Name: ***Program to Demonstrate Friend Function and Friend Class in C++***	**Date: 2026-04-16**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

Write a C++ program to demonstrate the use of **Friend Function** and **Friend Class**. Define a class named **Square** that represents a square shape. The class should contain **one private data member** to store the side length of the square. Implement the following:

1. A **friend function** that accesses the private data of the Square class and calculates the **area of the square**.
2. A **friend class** named **Rectangle** that accesses the same private data of the Square class and calculates the **area of a rectangle** formed using the same side as both its length and breadth.

Create objects of both classes and display:

- The area of the square using the **friend function**
- The area of the rectangle using the **friend class**

This program should clearly demonstrate how **friend functions** and **friend classes** can access private members of a class.

Input Format:

- The program reads **one integer** from standard input.
- The integer represents the **side length of the square.**

Output Format:

- Print the **area of the square** using the friend function.
- Print the **area of the rectangle** using the friend class.

Note:

- The input and output operations are already implemented.

Source Code:

Area.cpp

```
#include <iostream>
using namespace std;

// Forward declaration
class Rectangle;

class Square {
private:
   int side;
public:
   Square()
{
        side = 0;
}
    Square(int s)
{
        side = s;
}
friend int
calculateSquareArea(Square s);
friend class Rectangle;
};
// Friend function definition
int calculateSquareArea(Square s) {
return s.side*s.side;
}
class Rectangle {
   private:
     Square sq;
public:
Rectangle(Square s)
{
        sq = s;
}
int calculateRectangleArea()
{
        return sq.side*sq.side;
}
};
int main() {
    int side;
    cin >> side;

    Square sq(side);
```

```
    Rectangle rect(sq);
    cout << "Area of Square: " << calculateSquareArea(sq) << endl;
    cout << "Area of Rectangle: " << rect.calculateRectangleArea() <<
endl;
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
Area of Square: 16
Area of Rectangle: 16

Test Case - 2
User Output
7
Area of Square: 49
Area of Rectangle: 49

S.No: 11	Exp. Name: ***Sum of Array using inline function***	**Date: 2026-03-03**

Aim:

Write a C++ program to demonstrate the use of an **inline function**. Define an **inline function** named sumArray() that takes an integer array and its size as parameters and returns the **sum of all elements** in the array. Inside the main() function:

- Read the size of the array from the user
- Dynamically create an integer array of the given size
- Read the array elements from standard input
- Call the **inline function sumArray()** to calculate the sum
- Display the computed sum

Input Format:

- The first input is an integer n, representing the size of the array.
- The next n lines each contain one integer, representing the elements of the array.

Output Format:

- The output is a single integer, which is the sum of the elements of the array.

Note:

- Refer to the visible test cases for better understanding.

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

SumByInlineFunc.cpp

```
#include <iostream>
using namespace std;

// Inline function to calculate the sum of elements in an integer array
inline int sumArray(int arr[], int size) {
int sum = 0;
        for(int i = 0;i < size; i++) {
                sum += arr[i];
        }
        return sum;
}
int main() {
    int size;
    cout << "Enter the size of the array: ";
    cin >> size;

    int* arr = new int[size];

    cout << "Enter " << size << " integers:" << endl;
    for (int i = 0; i < size; ++i) {
        cout << "Enter integer " << i + 1 << ": ";
        cin >> arr[i];
    }

    int sum = sumArray(arr, size);
    cout << "Sum of array elements: " << sum << endl;

    delete[] arr;
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the size of the array:
5
Enter 5 integers:
Enter integer 1:
1

```
Enter integer 2:
2
Enter integer 3:
3
Enter integer 4:
4
Enter integer 5:
5
Sum of array elements: 15
```

Test Case - 2

User Output

```
Enter the size of the array:
4
Enter 4 integers:
Enter integer 1:
19
Enter integer 2:
14
Enter integer 3:
13
Enter integer 4:
12
Sum of array elements: 58
```

S.No: 12	Exp. Name: ***Class and Object - STUDENT Details in C++***	**Date: 2026-03-03**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

A college system needs to store and display basic details of a student, such as the student ID and marks. To manage this information efficiently, a class is used with member functions to input and display student details.

Create a class named **STUDENT** to store student information. The class should contain two data members:

- sid → Student ID
- smarks → Student Marks

The class should also include:

- A member function **GetData()** to read the values of the data members from the user
- A member function **ShowData()** to display the values of the data members

Create an object of the **STUDENT** class in the main() function and call the member functions to demonstrate the working of the class.

Input Format:

- The first input is an integer representing the **Student ID**, entered when the program prompts: **Enter Student ID: <<input>>**
- The second input is an integer representing the **Student Marks**, entered when the program prompts: **Enter Student Marks:<<input>>.**

Output Format:

- Display the **Student ID** in the following format: **Student ID: <value>**
- Display the **Student Marks** in the following format: **Student Marks: <value>**

Source Code:

Student.cpp

```
#include <iostream>
using namespace std;

class STUDENT {
private:
int sid;
int smarks;
public:
void GetData()
{
        cout<< "Enter Student ID: ";
        cin >> sid;

        cout<<"Enter Student Marks: ";
        cin >> smarks;
}
void ShowData()
{
        cout<<"Student ID: "<< sid << endl;
        cout<< "Student Marks: "<< smarks<< endl;
}
};

int main() {
    STUDENT s;
    s.GetData();
    s.ShowData();
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter Student ID:
10
Enter Student Marks:
95
Student ID: 10

Test Case - 2
User Output
`Enter Student ID:`
11
`Enter Student Marks:`
86
`Student ID: 11`
`Student Marks: 86`

S.No: 13	Exp. Name: ***Class and Object – EMPLOYEE Details in C++***	**Date: 2026-03-03**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

An organisation needs a simple program to store and display basic employee details such as employee ID, name, and salary. To achieve this, a class is used with member functions to input and display employee information. Create a class named **EMPLOYEE** to store employee information. The class should contain **three data members**:

- eid → Employee ID (integer)
- ename → Employee Name (character array)
- esalary → Employee Salary (float)

The class should also include **two member functions**:

- GetDetails() → to read the values of all data members
- PrintDetails() → to display the employee details

Create an object of the **EMPLOYEE** class in the main() function and call the member functions to demonstrate the working of the class.

Input Format:

- The first input is an integer representing the **Employee ID**, entered when the program prompts: **Enter Employee ID:**
- The second input is a string representing the **Employee Name**, entered when the program prompts: **Enter Employee Name:**
- The third input is a floating-point value representing the **Employee Salary**, entered when the program prompts: **Enter Employee Salary:**

Output Format

- Display the **Employee ID** in the format: **Employee ID: <value>**
- Display the **Employee Name** in the format: **Employee Name: <value>**
- Display the **Employee Salary** in the format: **Employee Salary: <value>**

Note:

- eid must be of type int
- ename must be a character array
- esalary must be of type float
- Input and output are handled using member functions

Source Code:

EMPLOYEE.cpp

```
#include<iostream>
using namespace std;

class EMPLOYEE {
private:
int eid;
char ename[100];
float esalary;
public:
        void GetDetails() {
                cout<<"Enter Employee ID: ";
                cin >> eid;

                cout<<"Enter Employee Name: ";
                cin >> ename;

                cout<<"Enter Employee Salary: ";
                cin >> esalary;
        }

void printDetails()
{
        cout<< "Employee ID: "<< eid << endl;
        cout<<"Employee Name: "<< ename << endl;
        cout<<"Employee Salary: " << esalary << endl;
}
};
int main()
{
        EMPLOYEE e;
        e.GetDetails();
        e.printDetails();
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter Employee ID:

101
Enter Employee Name:
Ramesh
Enter Employee Salary:
35000.50
Employee ID: 101
Employee Name: Ramesh
Employee Salary: 35000.5

Test Case - 2
User Output
Enter Employee ID:
205
Enter Employee Name:
Anitha
Enter Employee Salary:
42000
Employee ID: 205
Employee Name: Anitha
Employee Salary: 42000

S.No: 14	Exp. Name: ***Function Overloading***	**Date: 2026-03-04**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

A geometry-based application needs to calculate the areas of different shapes, such as a rectangle, a circle, and a triangle. To simplify the design and reuse the same function name for different calculations, **function overloading** is used. Write a C++ program to demonstrate **function overloading**. Create multiple functions with the **same function name** but **different parameter lists** to calculate the area of different geometric shapes. The program should calculate the area of:

- A **rectangle** using its length and breadth
- A **circle** using its radius
- A **triangle** using its base and height

Use **function overloading** to implement all the above area calculations.

Input Format:

- **First line:** Two integers representing the **length and breadth of the rectangle** *(entered in the format: length breadth)*
- **Second line:** One integer representing the **radius of the circle** *(entered as: radius)*
- **Third line:** Two integers representing the **base and height of the triangle** *(entered in the format: base height)*

Output Format:

- Print the **area of the rectangle.**
- Print the **area of the circle.**
- Print the **area of the triangle.**
- Each value should be printed on a **separate line** and formatted to **two decimal places**.

Note:

- Use the value **PI = 3.14159** for calculating the area of the circle, and use **0.5** as the value of $\frac{1}{2}$ while calculating the area of the triangle.
- Output must be displayed using **fixed-point notation with two decimal precision.**
- The program must demonstrate **function overloading.**

Source Code:

functOverload.cpp

```
#include <iostream>
#include <iomanip>
#define PI 3.14159

using namespace std;

// Rectangle area
double calculateArea(int length, int breadth) {
    return length * breadth;
}

// Circle area
double calculateArea(int radius) {
        return PI * radius * radius;
}

// Triangle area
double calculateArea(int base, int height, bool isTriangle) {
    return 0.5 * base * height;
}

int main() {
    int length, breadth, radius, base, height;

    // Input for rectangle
    cin >> length >> breadth;

    // Input for circle
    cin >> radius;

    // Input for triangle
    cin >> base >> height;

    cout << fixed << setprecision(2);

    cout << calculateArea(length, breadth) << endl;
    cout << calculateArea(radius) << endl;
    cout << calculateArea(base, height, true) << endl;

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
10 5
7
6 4
50.00
153.94
12.00

Test Case - 2
User Output
8 6
5
10 4
48.00
78.54
20.00

ID: 2503051050907
2025-2029-2A13-3
Parul University

S.No: 15	Exp. Name: ***Implement Single Inheritance in C++***	**Date: 2026-03-04**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:
Write a program to implement single inheritance in C++. Create a base class **Student** with attributes for roll number and student name.

Derive a class **Marks** from **Student** that includes attributes for three marks, total marks, and average marks. The program should allow the user to input the student's roll number, name, and marks, and then display the student's information along with the total and average marks.

Class: Student
Data Members:
- **rollno** - integer to store the student roll number
- **sname** - string to store the student name

Member Function:
- **setdata(int r, string name) -** Assigns the roll number and student name to the data members.

Class: Marks (Derived from **Student)**
Data Members:
- **m1, m2, m3** – integers to store marks of three subjects
- **total** – integer to store the total marks
- **avg** – float to store the average marks

Member Functions:
- **setmarks(int mark1, int mark2, int mark3)** -Assigns marks, calculates total and average.
- **show()** - Displays the student details, marks, total, and average.

Input Format:
- The first line of input is the roll number (integer) of the student.
- The second line of input is the student's name (string).
- The third line represents the marks for three subjects (three integers) separated by spaces.

Output Format:
The details of the student should be displayed in the following order,
each on a new line:
- Roll number
- Student name
- Marks for subject 1
- Marks for subject 2
- Marks for subject 3

- Total marks
- Average marks formatted to four decimals

Note: Refer to the visible test cases to strictly match the input/output formats.

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

Inheritance.cpp

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Student {
  protected:
int rollno;
string sname;

public:
void setdata(int r,string name){
            rollno = r;
            sname =  name;
}
};

class Marks : public Student {
private:
int m1,m2,m3;
int total;
float avg;

public:
void setmarks(int mark1, int mark2, int mark3) {
        m1 = mark1;
        m2 = mark2;
        m3 = mark3;

        total = m1 + m2 + m3;
        avg = total / 3.0;
}
void show()
{
        cout<< rollno << endl;
        cout<< sname << endl;
        cout<< m1 << endl;
        cout<< m2 << endl;
        cout << m3 << endl;
        cout<< total << endl;
        cout<< fixed << setprecision(4) << avg<<endl;
}
```

```
};

int main() {
    Marks mar;
    int rollno, m1, m2, m3;
    string sname;

    // User input
    cin >> rollno;
    cin.ignore(); // To ignore the newline character after the integer
input
    getline(cin, sname);
    cin >> m1 >> m2 >> m3;

    // Setting data and marks
    mar.setdata(rollno, sname);
    mar.setmarks(m1, m2, m3);

    // Displaying the information
    mar.show();

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2
Jane Smith
78 82 88
2
Jane Smith
78
82
88
248
82.6667

Test Case - 2
User Output
1
John Doe
85 90 95
1
John Doe
85
90
95
270
90.0000

S.No: 16	Exp. Name: ***Implement Multilevel Inheritance in C++***	**Date: 2026-04-16**

Aim:

Write a C++ program to demonstrate multilevel inheritance.

- Create a class **base** that contains an integer variable x and a member function **getdata()** to read its value.
- Derive a class **derive1** from **base** that contains an integer variable y and a member function **readdata()** to read its value.
- Further, derive a class **derive2** from **derive1** that contains an integer variable z and provides two member functions:
    - **indata()** to read the value of z, **product()** to calculate and display the product of x, y, and z
- The program should allow the user to enter values for x, y, and z, and then display their product.

Input Format:

- The first input is an integer representing the value of x.
- The second input is an integer representing the value of y.
- The third input is an integer representing the value of z.

Output Format:

- Print a single integer representing the product of x, y, and z.

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

MulInheritance.cpp

```
#include <iostream>
using namespace std;

class base {
        protected:
int x;
public:
void getdata()
{
        cin>>x;
}
};

class derive1 : public base{
protected:
int y;
public:
void readdata()
{
        cin >> y;
}

};

class derive2 : public derive1{
int z;
public:
void indata(){
cin >>z;
}
void product()
{
        cout << x * y * z << endl;
}
};

int main() {
    derive2 a;
    a.getdata();
    a.readdata();
    a.indata();
    a.product();
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2
3
4
24

Test Case - 2
User Output
5
6
7
210

ID: 2503051050907
2025-2029-2A13-3
Parul University

S.No: 17	Exp. Name: ***Multiple Inheritance in C++***	**Date: 2026-04-23**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

A university wants to maintain students' personal details along with their academic and course-related information. To efficiently model this system, **multiple inheritance** is used so that a student's personal, academic, and course details can be combined in a single class. Write a C++ program to demonstrate the use of **multiple inheritance**.

Create four classes as described below:

- **Person**
  - Stores personal details such as name, address, city, and age.
- **Student**
  - Inherits from the Person class.Stores student-specific details such as student ID and major (branch).
- **Course**
  - Stores course-related details such as course hours and subject.
- **StudentCourse:**
- Inherits from both Student and Course classes.Combines personal, student, and course details.

Each class should contain appropriate member functions to read and display the data. Create an object of the StudentCourse class to input and display all the details.

Input Format:

The input is provided in the following order:

- A string representing the **student name**
- A string representing the **address**
- A string representing the **city**
- An integer representing the **age**
- An integer representing the **student ID**
- A string representing the **major (branch)**
- An integer representing the **course hours**
- A string representing the **subject.**

Output Format:

Display the following details in order, each on a new line:

- Name: followed by the student's name
- Address: followed by the address
- City: followed by the city
- Age: followed by the age
- Student ID: followed by the student ID
- Major: followed by the major
- Course Hours: followed by the course hours
- Subject: followed by the subject

Note: Refer to the visible test cases for better understanding.

Source Code:

StudentCourse.cpp

```
#include<iostream>
using namespace std;

class Person
{
//write your code here
  protected:
string name, address, city;
int age;
public:
void getPersonData() {
        getline(cin,name);
        getline(cin,address);
        getline(cin,city);
        cin>>age;
}
  void showPersonData() {
          cout<<"Name:"<<name<<endl;
          cout<<"Address:"<<address<<endl;
          cout<<"City:"<<city<<endl;
          cout<<"Age:"<<age<<endl;
  }
};

class Student : public Person
{

//write your code here
  protected:
int sid;
string major;
public:
void getStudentData() {
        cin>>sid;
        cin.ignore();
        getline(cin,major);
}
  void showStudentData() {
          cout<<"Student ID:"<<sid<<endl;
          cout<<"Major:"<<major<<endl;
  }
```

Parul University 2025-2029-2A13-3 ID: 2503051050907 Page No: 47

```
};

class Course
{

//write your code here
  protected:
int hours;
string subject;
public:
void getCourseData() {
        cin>>hours;
        cin.ignore();
        getline(cin,subject);
}
  void showCourseData() {
          cout<<"Course Hours:"<<hours<<endl;
          cout<<"Subject:" <<subject<<endl;
  }





};

class StudentCourse : public Student, public Course
{

//write your code here
public:
void getStudentCourseDetails() {
        getPersonData();
        getStudentData();
        getCourseData();
}
 void displayStudentCourseDetails() {
         showPersonData();
         showStudentData();
         showCourseData();
 }
```

```
};

int main()
{
    StudentCourse sc;
    sc.getStudentCourseDetails();
    sc.displayStudentCourseDetails();
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Ravi
MG Road
Hyderabad
21
1023
CSE
120
Data Structures
Name:Ravi
Address:MG Road
City:Hyderabad
Age:21
Student ID:1023
Major:CSE
Course Hours:120
Subject:Data Structures

Test Case - 2
User Output
Anitha
Park Street
Bangalore

22
5304
ECE
90
Digital Electronics
Name:Anitha
Address:Park Street
City:Bangalore
Age:22
Student ID:5304
Major:ECE
Course Hours:90
Subject:Digital Electronics

S.No: 18	Exp. Name: ***Hierarchical Inheritance - C++***	**Date: 2026-03-05**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

Write a C++ program to demonstrate hierarchical inheritance. Create a base class **Shape** that contains a protected integer variable **side** and two public member functions:

- **setSide(int s)** to assign a value to side
- **getSide()** to return the value of side

Derive two classes from **Shape**:

- **Square** — contains a member function **getArea()** that calculates and returns the area of a square.
- **Cube** — contains a member function **getVolume()** that calculates and returns the volume of a cube.

The program should accept the side length from the user, store it using the **setSide()** function, and then display the area of the square and the volume of the cube using the respective derived class objects.

Input Format:

- The input consists of a single integer value entered in the following format: **Side length: <side>**
- Where <side> is the length of the side.

Output Format:

- The output must be displayed in the following order:

```
Area of square: <area>
Volume of cube: <volume>
```

Constraints:

- $1 \leq \text{side} \leq 1000$

Source Code:

hierarchical_inheritance.cpp

```
#include <iostream>
using namespace std;
class Shape {
protected:
int side;

public:
void setSide(int s) {
        side = s;
}
int getSide()
{
        return side;
}
};

// First derived class
class Square : public Shape {
public:
int getArea() {
        return side*side;
}
};

// Second derived class
class Cube : public Shape{
public:
  int getVolume() {
          return side*side*side;
  }

};

int main() {
    int s;
    cout << "Side length: ";
    cin >> s;

    Square sq;
    Cube cu;

    sq.setSide(s);
    cu.setSide(s); // Use setter instead of accessing `side` directly
```

```
    cout << "Area of square: " << sq.getArea() << endl;
    cout << "Volume of cube: " << cu.getVolume() << endl;

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Side length:
12
Area of square: 144
Volume of cube: 1728

Test Case - 2
User Output
Side length:
4
Area of square: 16
Volume of cube: 64

S.No: 19	Exp. Name: ***Virtual Base Class in C++***	**Date: 2026-04-25**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

Write a C++ program to demonstrate the use of a virtual base class. Create a base class called **Person** with members for **name** and **age**. Derive two classes, **Student** and **Teacher**, from **Person** using **virtual inheritance**. Then create a **TA** (Teaching Assistant) class that inherits from both Student and Teacher.

Class and Methods:

Class Person:

- **void getPersonDetails():** Reads the name (string) and age (int) from user input.
- **void showPersonDetails():** Displays name and age.

Class Student (inherits virtually from Person):

- **void getStudentDetails():** Reads student ID (string).
- **void showStudentDetails():** Displays student ID.

Class Employee (inherits virtually from Person):

- **void getEmployeeDetails():** Reads employee ID (string).
- **void showEmployeeDetails():** Displays employee ID.

Class Intern (inherits from both Student and Employee):

- **void getDetails():** Calls all relevant get functions to collect complete TA data.
- **void showDetails():** Displays all collected data in the required format.

Input Format:

- The first line of input contains a string that represents the name of the teaching assistant.
- The second line of input contains an integer that represents the age.
- The third line of input contains a string that represents the student ID.
- The fourth line of input contains a string that represents the employee ID.

Output Format:

- Print each of the following on separate lines:

```
Name: <name>
Age: <age>
Student ID: <student_id>
Employee ID: <employee_id>
```

Source Code:

virtual_base_class.cpp

```
#include <iostream>
using namespace std;

class Person {

        //write your code here..
        protected:
string name;
int age;
public:
void getPersonDetails() {
        getline(cin,name);
        cin>>age;
        cin.ignore();
}
};

class Student : virtual public Person {

        //write your code here..
        protected:
string studentID;
public:
void getStudentDetails() {
        getline(cin,studentID);
}
};

class Employee : virtual public Person {

        //write your code here....
        protected:
string employeeID;
public:
void getEmployeeDetails() {
        getline(cin,employeeID);
}
};

class Intern : public Student, public Employee {

        //write your code here..
        public:
void getDetails() {
```

```
        getPersonDetails();
        getStudentDetails();
        getEmployeeDetails();
}
void showDetails() {
        cout<<"Name: "<<name<<endl;
        cout<<"Age: "<<age<<endl;
        cout<<"Student ID: "<<studentID<<endl;
        cout<<"Employee ID: "<<employeeID<<endl;
}
};

int main() {
    Intern intern;
    intern.getDetails();
    intern.showDetails();
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Alice JohnSon
22
S123
E456
`Name: Alice JohnSon`
`Age: 22`
`Student ID: S123`
`Employee ID: E456`

Test Case - 2
User Output
Bob Smith
25
ST789
EMP987

Name: Bob Smith
Age: 25
Student ID: ST789
Employee ID: EMP987

S.No: 20	Exp. Name: ***Default Constructor in C++***	**Date: 2026-03-19**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

Write a C++ program to demonstrate the use of a **default constructor**. Create a class named **Demo** that contains a **default constructor**. When an object of the class is created, the default constructor should automatically execute and display a message indicating that the constructor has been called. Create an object of the class in the main() function to invoke the default constructor.

Input Format:

- No input is required for this program.

Output Format:

- Display the following message: **This is a default constructor.**

Source Code:

constructors.cpp

```
#include <iostream>
using namespace std;

class Demo {
public:
Demo(){
cout<<"This is a default constructor";
}
};

int main() {
    Demo d;   // Default constructor is called automatically
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
This is a default constructor

S.No: 21	Exp. Name: ***Parameterized Constructor in C++***	**Date: 2026-04-23**

Aim:

Write a C++ program to demonstrate the use of a **Parameterized Constructor**. Create a class named **Person** with the following features:

- Two public data members:
    - name – to store the person's nameage – to store the person's age
- A **parameterised constructor** that initialises the name and age using values passed as arguments
- A **destructor** that displays a message when a Person object is deleted

In the main() function:

- Read the name and age of the person from the user
- Create an object of the Person class using the **parameterised constructor**
- Display the name and age of the person
- Allow the destructor to execute when the program ends

Input Format:

- The first line contains a string representing the **name of the person**
- The second line contains an integer representing the **age of the person**

Output Format:

- Display the person's details in the format: **<name> is <age> years old.**
- On program termination, display the destructor message in the format: **<name> has been deleted**

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

parameterised.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Person {
public:
    string name;
    int age;

    // Parameterized constructor
    Person(string n, int a) {
                name = n;
        age = a;
    }

    // Destructor
    ~Person() {
        // write your code here
      cout<< name<< " has been deleted"<< endl;
    }
};

int main() {
    string name;
    int age;

    cin >> ws;
    getline(cin, name);
    cin >> age;

    Person p(name, age);

    // write your code here to display output
   cout<< name<<" is " << age << " years old" << endl;
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

Parul University 2025-2029-2A13-3 ID: 2503051050907 Page No: 60

Amith
22
`Amith is 22 years old`
`Amith has been deleted`

Test Case - 2
User Output
Sumith
20
`Sumith is 20 years old`
`Sumith has been deleted`

S.No: 22	Exp. Name: ***Program to Demonstrate Copy Constructor***	**Date: 2026-04-25**

Aim:

Write a C++ program to demonstrate the use of a copy constructor. Create a class **Student** with the following private data members:

- int rollNumber
- string name

Implement:

- **Parameterised constructor** to initialise data members.
- **Copy constructor** to copy data from one object to another.
- A member function **display()** to print the details of a student.

Create an object using the parameterised constructor and then create another object using the copy constructor. Display the data of both objects.

Input Format:

- The first line of input contains an integer representing the roll number.
- The second line of input contains a string representing the name of the student.

Output Format:

- Display details of the original object followed by the copied object:

Source Code:

ID: 2503051050907
2025-2029-2A13-3
Parul University

copyconst.cpp

```
#include <iostream>
#include <string>
using namespace std;


//write your code here...
class Student {
private:
int rollNo;
string name;
public:
Student(int r, string n) {
        rollNo = r;
        name = n;
}
Student(const Student &s) {
        rollNo = s.rollNo;
        name = s.name;
}
void display() {
        cout<<"Roll Number: "<<rollNo<<endl;
        cout<<"Name: "<<name<< endl;
}
};

int main() {
    int roll;
    string studentName;

    // Input
    cin >> roll;
    cin.ignore(); // ignore newline
    getline(cin, studentName);

    // Create original object
    Student original(roll, studentName);

    // Create copied object using copy constructor
    Student copy = original;

    // Display both objects
    cout << "Original Student:" << endl;
    original.display();
```

```
    cout << "Copied Student:" << endl;
    copy.display();

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
101
Amith
Original Student:
Roll Number: 101
Name: Amith
Copied Student:
Roll Number: 101
Name: Amith

Test Case - 2
User Output
102
Suresh
Original Student:
Roll Number: 102
Name: Suresh
Copied Student:
Roll Number: 102
Name: Suresh

S.No: 23	Exp. Name: ***Use of this Pointer - C++***	**Date: 2026-04-23**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

Write a C++ program to demonstrate the use of a **pointer to an object**. Create a class **Student** with the following members:

- Private data members: **name** (string) and **age** (int).
- A member function **setData(string name, int age)** to initialise the data.
- A member function **showData()** to display the name and age.

In the **main()** function:

- Create an object of the **Student** class.
- Create a pointer to this object.
- Read the name and age from the user.
- Use the pointer to call the member functions to set and display the data.

Input Format:

- A string representing the name.
- An integer representing the age.

Output Format:

- Print each of the following on separate lines:

```
Name: <name>
Age: <age>
```

Source Code:

ThisPointer.cpp

```
#include <iostream>
using namespace std;

class Student {

        //write your code here..
        private:
string name;
int age;
public:
void setData(string name, int age) {
        this->name=name;
        this->age=age;
}
void showData() {
        cout<<"Name: "<<name<<endl;
        cout<<"Age: "<< age<<endl;
}

};

int main() {
    Student s;
    string name;
    int age;

    getline(cin, name);
    cin >> age;

    s.setData(name, age);
    s.showData();

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Eve

21
`Name: Eve`
`Age: 21`

Test Case - 2
User Output
Frank
24
`Name: Frank`
`Age: 24`

S.No: 24	Exp. Name: ***Pointer to Derived Class - Single Inheritance***	**Date: 2026-04-23**

ID: 2503051050907
2025-2029-2A13-3
Parul University

Aim:

Write a C++ program to demonstrate how a pointer to a derived class object can be used to access both base and derived class members.

Create a base class named **Employee** with:

- Protected data members: **empName** (string), and **empID** (int).
- A member function **setEmployee(string name, int id)** to set the data.
- A member function, **displayEmployee()** to display the data.

Create a derived class named **Manager** with:

- A private data member: **department** (string).
- A member function **setManager(string name, int id, string dept)** that uses the base class method to set the name and ID and also sets the department.
- A member function **displayManager()** that displays employee details and department.

In the **main()** function:

- Read employee name, ID, and department from the user.
- Create a pointer to a **Manager** object.
- Use the pointer to call **setManager()** and **displayManager().**

Input Format:

- A string representing the employee's name.
- An integer representing the employee ID.
- A string representing the department.

Output Format:

```
Name: <empName>
ID: <empID>
Department: <department>
```

Source Code:

PointerToDerivedClass.cpp

```
#include <iostream>
using namespace std;

class Employee {

        //write your code here..
        protected:
string name;
int id;
public:
void setEmployee(string n, int i){
        name=n;
        id = i;
}
void displayEmployee() {
        cout<<"Name: "<<name<<endl;
        cout<<"ID: "<<id<<endl;
}
};

class Manager : public Employee {

        //write your code here..
        private:
string department;
public:
void setManager(string n,int i, string d) {
        setEmployee(n,i);
        department=d;
}
void displayManager() {
        displayEmployee();
        cout<<"Department: "<<department<<endl;
}
};

int main() {
    string name, dept;
    int id;

    getline(cin, name);
    cin >> id;
    cin.ignore();
    getline(cin, dept);
```

```
    Manager m;
    Manager* ptr = &m;

    ptr->setManager(name, id, dept);
    ptr->displayManager();

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Rahul Sharma
101
Sales
`Name: Rahul Sharma`
`ID: 101`
`Department: Sales`

Test Case - 2
User Output
Meera Patel
202
Finance
`Name: Meera Patel`
`ID: 202`
`Department: Finance`

Parul University 2025-2029-2A13-3 ID: 2503051050907 Page No: 70